



# Form

# Génération d'un formulaire

**Le workflow recommandé par Symfony lorsque l'on travaille avec des formulaires est le suivant :**

- 1) Construire le formulaire dans un contrôleur Symfony ou en utilisant une classe de formulaire dédiée ;
- 2) Rendre le formulaire dans un modèle pour que l'utilisateur puisse l'éditer et le soumettre ;
- 3) Traiter le formulaire pour valider les données soumises, les transformer en données PHP et en faire quelque chose (par exemple, les persister dans une base de données).

```
php bin/console make:form
```

Cela va générer une classe que l'on pourra modifier pour ajouter les champs que l'on souhaite rendre modifiables dans le formulaire.

Les formulaires sont générés dans le dossier /src/Form

```
use Symfony\Component\Form\Extension\Core\Type\TextType;
use Symfony\Component\Form\AbstractType;
use Symfony\Component\Form\FormBuilderInterface;

class RegistrationFormType extends AbstractType {
    public function buildForm(FormBuilderInterface $builder, array $options): void {
        $builder
            ->add('name', TextType::class)
            ->add('email', EmailType::class)
            ->add('password', PasswordType::class);
    }
}
```

**Types de champs fréquemment utilisés :**

- TextType
- TextareaType
- EmailType
- PasswordType
- IntegerType
- NumberType
- MoneyType
- DateType
- DateTimeType
- TimeType
- ChoiceType
- CheckboxType
- RadioType
- FileType
- HiddenType
- CollectionType
- SubmitType

# Affichage et traitement d'un formulaire

```
<?php
// src/Controller/TaskController.php
namespace App\Controller;

use App\Form\Type\RegistrationType;
// ...

0 references | 0 implementations
class TaskController extends AbstractController
{
    0 references | 0 overrides
    public function new(): Response
    {
        $registration = new Registration();
        $registration->setName('Write a blog post');

        $form = $this->createForm(RegistrationType::class, $registration);

        return $this->render('task/new.html.twig', [
            'form' => $form,
        ]);
    }
}
```

La classe form contient toutes les instructions nécessaires pour créer le formulaire.

Dans les contrôleurs qui s'étendent à partir du contrôleur AbstractController, utilisez createForm()



# Affichage et traitement d'un formulaire

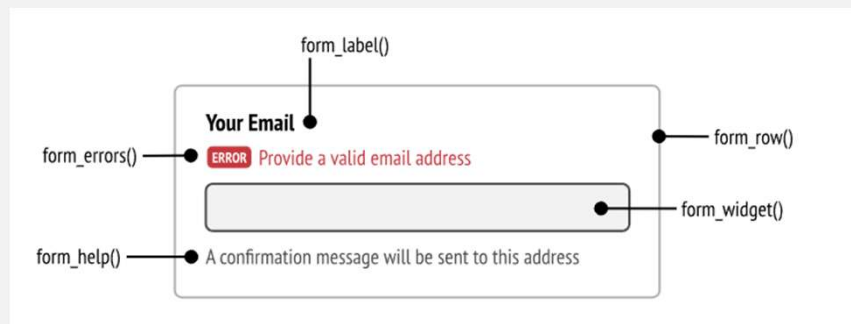
```
{{ form_start(form) }}  
  <div class="my-custom-class-for-errors">  
    {{ form_errors(form) }}  
  </div>  
  
  <div class="row">  
    <div class="col">  
      {{ form_row(form.name) }}  
    </div>  
    <div class="col">  
      {{ form_row(form.email) }}  
    </div>  
    <div class="col">  
      {{ form_row(form.password) }}  
    </div>  
  </div>  
{{ form_end(form) }}
```

Dans twig, il existe des balises pour afficher les champs issus du formulaire

La fonction `form_row()` produit l'ensemble du contenu du champ, y compris l'étiquette, le message d'aide, les éléments HTML et les messages d'erreur.

Tout cela peut être personnalisé à l'aide d'autres fonctions Twig, comme l'illustre le diagramme suivant :

```
<div class="form-control">  
  {{ form_label(form.name) }}  
  {{ form_widget(form.dueDate) }}  
  
  <small>{{ form_help(form.name) }}</small>  
  
  <div class="form-error">  
    {{ form_errors(form.name) }}  
  </div>  
</div>
```



# Affichage et traitement d'un formulaire

```
<?php

namespace App\Controller\;

use App\Form\UserFormType;
use Doctrine\ORM\EntityManagerInterface;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\Validator\Constraints\Length;

/**
 * Class PayrollController
 * @package App\Controller\Payroll
 *
 * is_granted('ROLE_SOCIETE')
 */
class PayrollController extends AbstractController
{
    #[Route...]
    public function new(Request $request, EntityManagerInterface $entityManager):Response
    {
        $user = new User();

        $form = $this->createForm(UserFormType::class, $user);

        $form->handleRequest($request);

        if($form->isSubmitted() && $form->isValid()) {
            $entityManager->persist($user);
            $entityManager->flush();
        }

        return $this->render('user/new', [
            'form' => $form
        ]);
    }
}
```

## Pour enregistrer en base de données, il faut :

- Récupérer le formulaire posté via `handleRequest`
- Vérifier s'il est soumis et valide
- Enregistrer l'entité en base

## La fonction `handleRequest`

- 1) Détecte si le formulaire a été soumis
  - Si la méthode HTTP est POST (ou PUT, PATCH...), Symfony va analyser les données de la requête.
  - Il vérifie si les données soumises contiennent les champs du formulaire.
- 2) Remplit automatiquement l'objet lié
  - Symfony va hydrater l'objet passé lors de `createForm(...)`

# Validation avec asserts

```
<?php
use Symfony\Component\Validator\Constraints as Assert;

0 references | 0 implementations
class User {
    #[Assert\NotBlank]
    #[Assert\Length(min: 3, max: 50)]
    0 references
    private string $name;

    #[Assert\NotBlank]
    #[Assert\Email]
    0 references
    private string $email;

    #[Assert\NotBlank]
    #[Assert\Length(min: 6)]
    0 references
    private string $password;
}
```

Dans un formulaire, il est possible d'ajouter des asserts

Un assert permet d'imposer une règle à un champ dans un formulaire

## Liste des contraintes courantes :

- NotBlank – ne doit pas être vide
- NotNull – ne doit pas être null
- Length(min, max)
- Email, Url
- Regex(pattern=...)
- Choice(choices=...)
- Range(min, max)
- Positive, Negative, PositiveOrZero, NegativeOrZero
- IsTrue, IsFalse
- File, Image – pour upload
- Date, DateTime, Time
- Count(min, max) – pour des tableaux ou collections
- Callback – pour des validations personnalisées
- UniqueEntity – validation au niveau base de données

Si un assert est invalide dans le formulaire, alors symfony va retourner false dans `$form->isValid()` et afficher un message d'erreur dans la vue twig

<https://symfony.com/doc/current/validation.html>

# TP - Formulaires

## Étape 1 : Créer les entités

Créer les entités suivantes avec la commande `make:entity` :

### Product :

- name (string, 255)
- price (float)
- stock (int)

### Client :

- name (string)
- email (string)

### Commande :

- client (ManyToOne vers Client)
- product (ManyToOne vers Product)
- quantity (int)
- createdAt (datetime)

### Dans vos entités, utilisez les asserts :

- NotBlank – ne doit pas être vide
- NotNull – ne doit pas être null
- Length(min, max)
- Positive

## Étape 2 : Créer les formulaires

Créer les formulaires avec la commande `make:form` :

### ProductType :

- name
- price
- stock

### CommandeType :

- client (EntityType)
- product (EntityType)
- quantity

# TP - Formulaires

## Étape 3 : Enregistrement en base de données

Dans chaque contrôleur, créer une méthode 'create' avec traitement du formulaire

Exemple :

```
<?php

$product = new Product();
$form = $this->createForm(ProductType::class, $product);
$form->handleRequest($request);

if ($form->isSubmitted() && $form->isValid()) {
    $em->persist($product);
    $em->flush();
    return $this->redirectToRoute('product_list');
}
```

Dans chaque contrôleur, créer aussi une méthode 'update' avec traitement du formulaire

## Étape 4 : Affichage de toutes les entités

Créer une méthode 'index' dans chaque contrôleur pour afficher les entités :

Exemple :

```
<?php

public function index(ProductRepository $repo): Response {
    return $this->render('product/index.html.twig', [
        'products' => $repo->findAll(),
    ]);
}
```

```
...
<h1>Liste des produits</h1>
<ul>
{% for product in products %}
    <li>{{ product.name }} - {{ product.price }}€ (Stock: {{ product.stock }})</li>
{% endfor %}
</ul>
```

En haut de chaque page, ajouter 1 lien pour :

- Créer un nouveau client ou un nouveau produit ou une nouvelle commande
- Modifier chaque élément sur une ligne <li>